Two Paths in Quantum Software: Circuit Synthesis, AI Agents, and Platform Power

What do Classiq's high-level modeling and the Qiskit-CUDA-Q tool ecosystems automate differently? A conditional outlook grounded in specification correctness, qubit-operation tradeoffs, hardware integration, and barriers to adoption.

TECH REVIEW · October 25, 2026 · Quantum programming, compilers, and AI agents

Responsible editor: Hyun-Jung Kim · AI assistance: Codex · Evidence cutoff: 25 October 2026 · Production and verification



The template specifying an overall pattern and the shuttles combining different threads symbolize high-level synthesis and tool orchestration; the shared warp represents the constraints of execution and verification. The scene brings different design activities together on a common foundation, without declaring either company a winner. An AI-generated conceptual illustration, not an actual quantum device, circuit, or performance dataset.

1. The wrong question, answered faithfully by a machine

Imagine building a quantum program to find one correct answer among eight candidates. The answer is labeled by the three-bit string `101`. You ask an AI to write the program, but its circuit marks `111` as the answer. The syntax is valid, the operations are allowed by quantum mechanics, and the computation finishes normally. A better machine may simply find the wrong candidate more faithfully.

This small example offers a way into the competition in quantum software. Translating a natural-language request into code, implementing a mathematical model as a circuit, adapting that circuit to a particular device, and checking whether the result answers the original question are connected capabilities, but they do different jobs. To the user, it is one request to “run the calculation.” Between request and result lie several distinct responsibilities.

Classiq has developed an approach that expresses functions, quantum variables, and resource conditions in a high-level model, leaving implementation choices to a synthesis engine. Tool ecosystems centered on Qiskit and CUDA-Q combine supported algorithmic blocks with synthesis, compilation, and execution capabilities. An AI agent can sit in front of either approach. The competition therefore cannot be reduced to a single opposition between a new language and gate coding, or between proprietary and open-source software. [1-5]

The question that will determine platform power is less which interface writes more plausible code than which approach can keep reducing the total cost of turning the same objective into a verifiable result. High-level synthesis reduces the burden of implementation choices; tool ecosystems lower barriers to composition, modification, and deployment. Where the two paths overlap, interoperability grows. Where differences remain, comparative experiments are needed. This review examines those differences and their conditional implications through papers, documentation, and public code checked through October 25, 2026.

---

2. How long can a program retain “what to compute”?

In an ordinary program, an expression that adds two numbers is short. In a quantum program, it comes with further decisions: how many qubits represent the numbers, what sign and fractional precision they use, whether the result occupies fresh storage, and how the workspace used for intermediate calculations is restored. These decisions change the circuit's size and operation count.

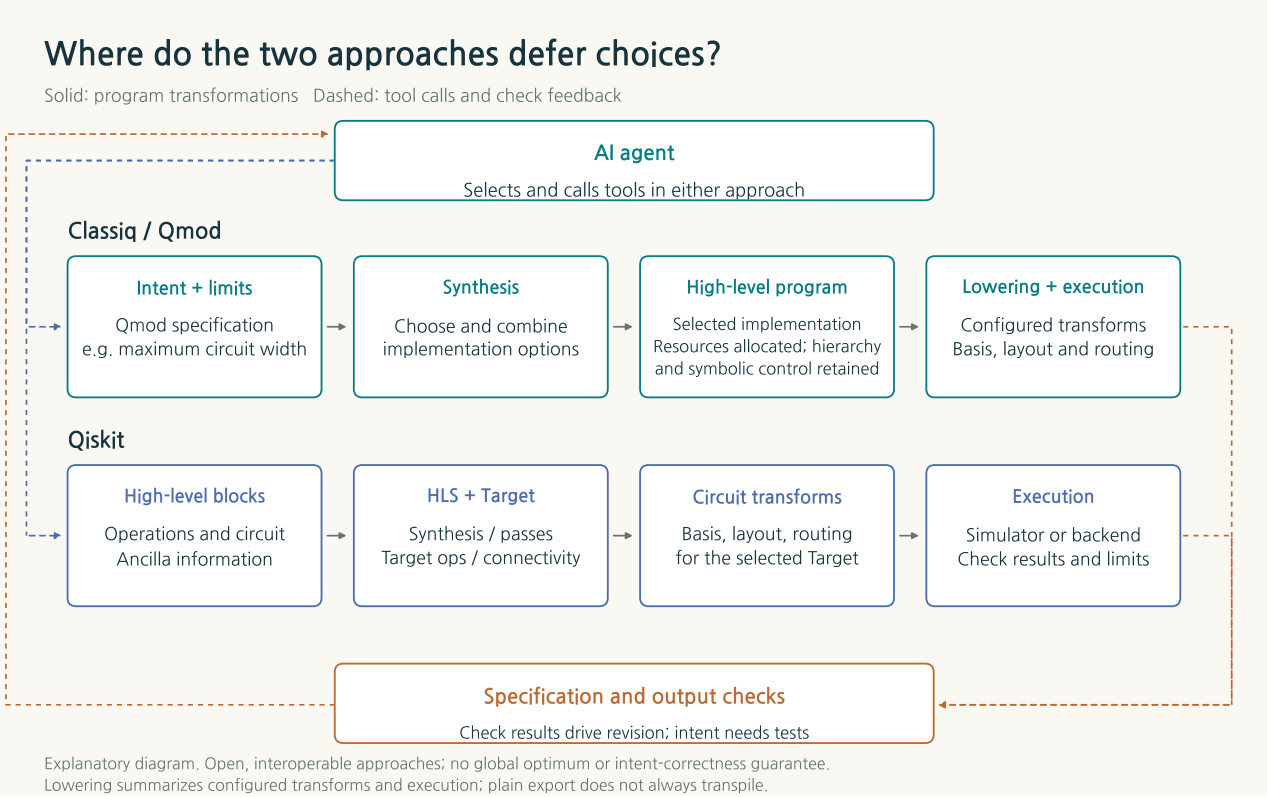
Qmod, Classiq's high-level quantum modeling language, retains quantum variables and functional relationships in the model. It can also be written using Python syntax, so users need not begin by learning a separate textual language. Writing a Python expression, however, does not mean that arbitrary Python computations execute unchanged on quantum states. Quantum numbers have finite bit widths and representable ranges, and the output width of an addition or the handling of overflow forms part of the program's meaning. [1,2]

Uncomputation—restoring temporary workspace to its original state after a function has been used—is another important example. Unlike erasing an intermediate value on paper, quantum computation must account for correlations left between the inputs and temporary registers. Qmod automates this task through particular type and dependency rules and structures for inverse operations. Reducing the implementation burden through such rules has real technical value. Extending their guarantees to the validity of the entire problem definition, or to success on every device, changes the scope of the claim. [2]

Qiskit also accepts high-level information. `HighLevelSynthesis` lowers supported abstract and annotated operations through synthesis methods, passing information about available auxiliary qubits to the synthesizers. For example, an MCX operation, which flips a target bit when all its control bits are 1, can remain a single high-

level object. The chosen decomposition depends on available workspace and evaluation criteria. Describing Qiskit as a tool that merely removes redundant gates from an already fully lowered circuit does not match its current implementation. [3]

That does not make the scope of automation identical in the two systems. The comparison concerns what meaning can be retained in the input, which choices are automated for supported operations, and how much users must manage program-wide resource allocation and connections between tools. Qmod's functional models and service-based synthesis, and Qiskit's extensible objects, plugins, and passes, offer overlapping but different design choices.



An explanatory reconstruction of roles identified in official documentation and pinned code. Agents can call tools along either path. This is neither an architecture diagram of every internal product stage nor a performance comparison, and it represents no guarantee of global optimality or fidelity to user intent. Classiq's current default flow separates higher-level program synthesis from subsequent target-specific transformation. [2-4]

What matters in the diagram is not the number of boxes, but which information survives until an implementation decision is made. Knowing that an operation is an addition, and knowing the available workspace, allows a tool to choose among different adders. If it receives only an already fixed sequence of low-level operations, recovering that higher-level intent may require extra work. Real tools operate at several levels, so synthesis and transpilation should not be assigned exclusively to one company each.

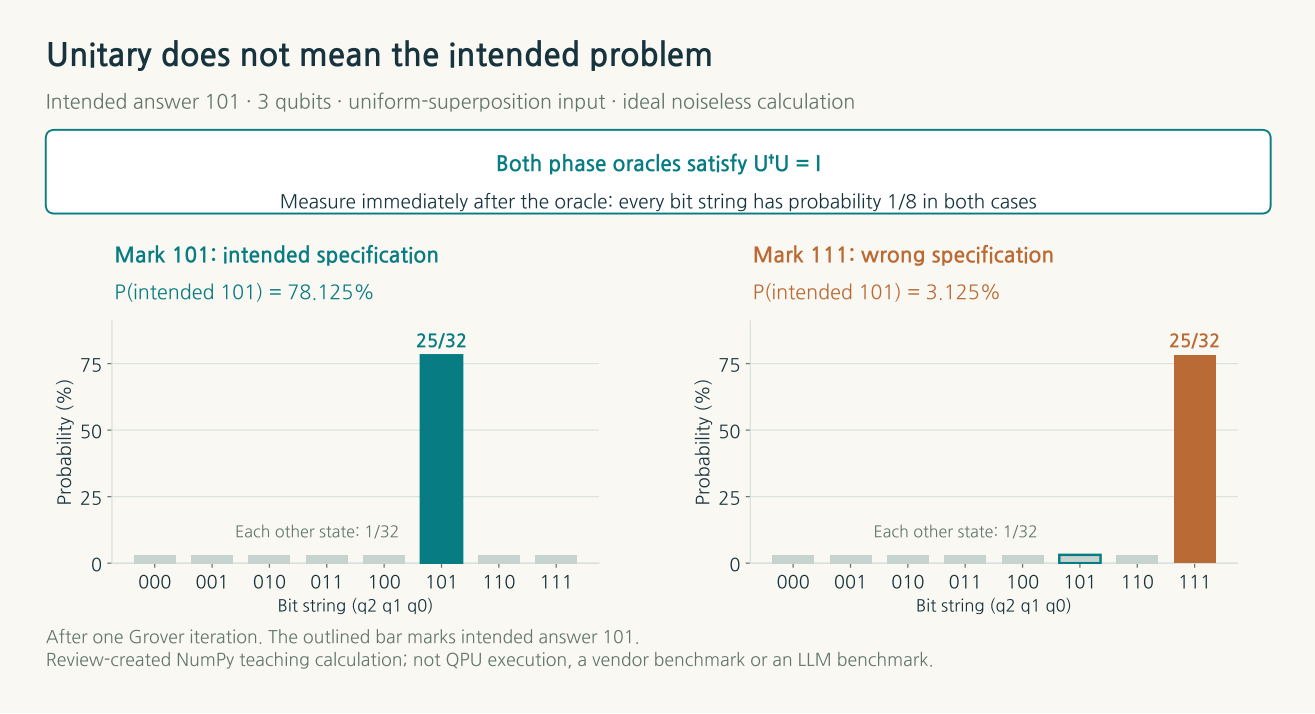
3. The questions a compiler check can answer

Consider the eight-candidate example more closely. The computational basis of three qubits runs from `000` to `111`. After preparing equal amplitudes for all candidates, we apply a phase oracle that puts a minus sign on one particular candidate. The oracle is the part that implements the algorithm's search condition as a quantum operation. [6]

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle.$$

Here, x denotes a candidate's three bits, and $f(x)$ is a function that returns 1 when the condition is satisfied and 0 otherwise. The phase oracle changes the sign of the amplitude of a matching candidate. Because probability is the squared magnitude of amplitude, measuring the bits immediately afterward still gives a probability of 1/8 for each of the eight candidates. That measurement distribution alone cannot distinguish whether the correct sign was changed.

For this review, we calculated the example exactly using an eight-dimensional NumPy statevector. The correct oracle marks `101`, while a deliberately incorrect oracle marks `111`. Both are unitary operations: they preserve the state's normalization. Following the phase oracle with one application of the diffusion operation—a reflection about the mean amplitude—gives the candidate marked by each oracle a probability of 25/32, and each remaining candidate a probability of 1/32. The probability of obtaining the intended answer, `101`, is **78.125%** with the correct specification and **3.125%** with the incorrect one. [6,7]



An ideal three-qubit statevector calculation independently executed for this review. Basis strings are ordered q2 q1 q0, and the intended answer is 101. Checks covered oracle unitarity, probability normalization, and the values 25/32 and 1/32. This is not a benchmark running Qiskit, Classiq, or an LLM, nor a physical-QPU result. [7]

The phase oracle followed by diffusion constitutes one Grover iteration here. Both outputs in the figure contain a tall bar. That is why the presence of a tall bar alone does not establish that the original problem was solved correctly. Even if a synthesis engine implements its input model faithfully, the result changes when the input f specifies a different problem. Conversely, a circuit that incorrectly implements a valid model presents an error for compilation or equivalence checking to address. The meaning of the problem, preservation through implementation, and accuracy of physical execution must each be checked.

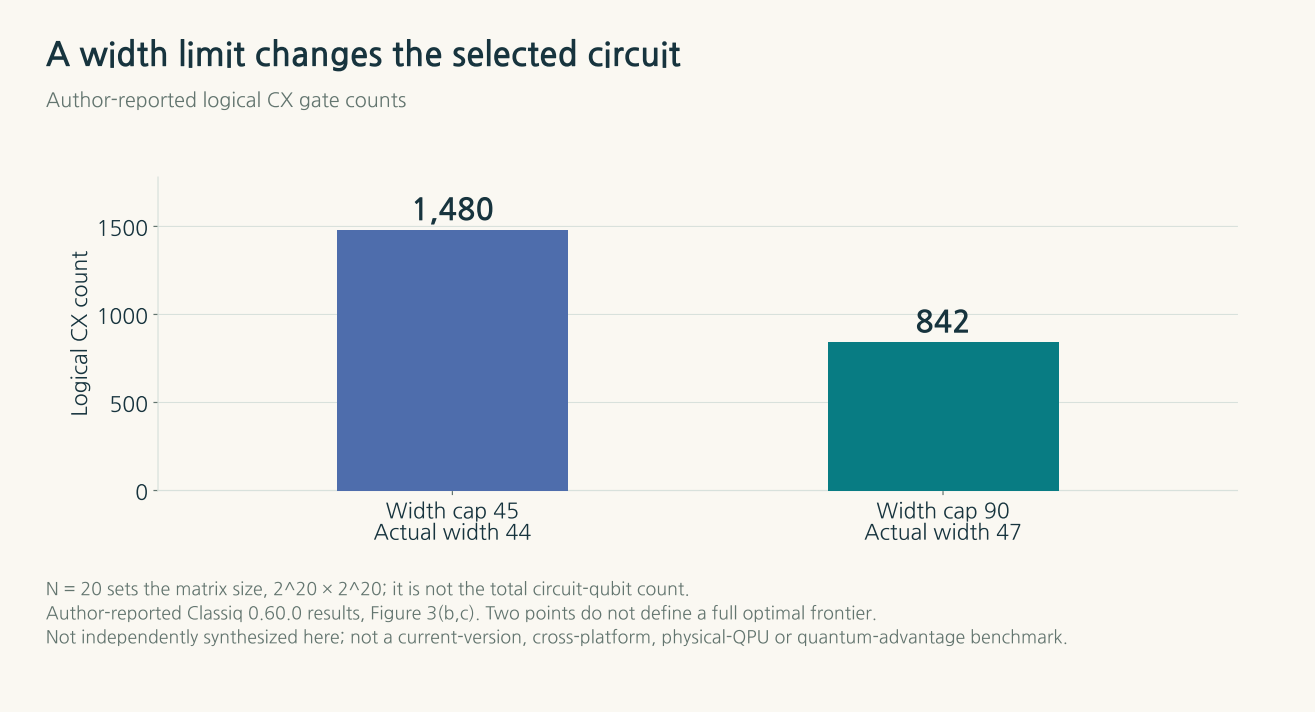
Classiq's automatic cleanup rules are likewise not an unconditional proof of the whole algorithm. Its documentation identifies boundaries around unchecked declarations and deallocation conditions, and its own Quantum Engineering Interface explicitly disclaims a guarantee that AI results are correct. Qiskit's Target is also a device model describing supported operations, connectivity, timing, and error properties—not a tool for judging the user's research question. This distinction establishes a fair starting point for comparing the two approaches, rather than singling out a defect in either one. [2,4,8]

---

4. Where synthesis makes a difference: borrowing space to reduce operations

The strength of synthesis extends beyond quickly writing a circuit whose implementation has already been decided. It lies in selecting an implementation that fits the resource conditions from several ways of performing the same operation. With fewer auxiliary qubits, intermediate values may need to be computed and uncomputed more often. Allowing more space can enable a different arrangement of operations. What counts as a good implementation therefore depends on which resource is being conserved.

A key metric is CX, the controlled-X or CNOT operation, which flips the target bit when the control bit is 1. Goldfriend and colleagues' synthesis preprint illustrates implementation choices through a matrix block-encoding example. Block encoding embeds the required matrix in part of a larger unitary operation. In Figure 3, $N = 20$ means that the matrix has dimensions $2^{20} \times 2^{20}$; it is not the total number of circuit qubits. With the objective of reducing CX count, the authors report an actual width of 44 qubits and 1,480 CX operations under a width cap of 45, and 47 qubits and 842 CX operations under a cap of 90. The selected adder implementation also changes. [9]



A new chart of the author-reported values in arXiv:2412.07372v2, Figure 3(b,c). These are logical-circuit compilation results using Classiq 0.60.0, not an independent run by this review. Width caps are distinguished from the number of qubits actually used. The two points represent neither a complete optimal frontier, a ranking of current products, physical speed, nor quantum advantage. [7,9]

The second implementation uses three more qubits and 638 fewer logical CX operations. The synthesis engine selected a different adder while leaving some of the permitted space unused. In this example, the space cap changed the range of available implementations, and the circuit ultimately selected used 47 qubits. The effect on execution time on a physical device depends on connectivity, operation times, noise, and error-correction costs.

Current Qiskit 2.6.1 also makes some resource-dependent choices of this kind. In the pinned code, the default MCX synthesis receives information about the availability of clean auxiliary qubits in the zero state and dirty auxiliary qubits whose states are unknown. It changes the priority of candidate implementations depending on whether the criterion is two-qubit operation count or T count. Clean workspace starts in a known zero state. Dirty workspace can be borrowed without knowing its state, but must be returned to the state it had before being borrowed. T is a gate that changes the relative phase by π/4 and is costly to implement in prominent error-correction schemes. Different decompositions can therefore be preferable depending on which count is minimized. This does not mean that optimal allocation across an entire program is automatically solved, but the comparison has already moved beyond simple gate-by-gate coding. [3]

Considering resource allocation globally is also different from proving a globally optimal solution. The synthesis paper's appendix explains that some search strategies can miss a possible optimum. The comparisons in that study used versions including Classiq 0.60.0 and Qiskit 1.2.4. They cannot be read as a current ranking against the latest Qiskit, CUDA-Q, and AI agents under matched conditions. [9]

Version differences also affect how the tools are used. The Classiq SDK examined at the cutoff was 1.33.1. Its current default flow preserves the synthesized higher-level program and separates device-specific gate and connectivity transformations into later stages. Default export skips additional transformation; target-specific transformation occurs during configured export, requests for post-transpilation metrics, and execution. General approximate metrics must also be distinguished from metrics obtained after target-specific transformation. Carrying an old paper and its configuration examples directly into an explanation of the current API obscures what was optimized. Comparisons need to fix the input model, compiler versions, objective function, target, and search time together. [2,10]

5. What agents do—and what they delegate

Can an LLM be made to assemble large Qiskit function blocks? Yes. Connections already exist that let agents call tools, read documentation, and construct or transform circuits. The existence of such a connection, however, must be distinguished from evidence that it automatically produces a correct algorithm.

Component	What it actually does	Questions a comparison must retain
MCP server	Exposes tools, resources, and call formats to an agent	Which synthesis engines, checkers, and execution services does it actually call?
AI skill	Supplies task-specific instructions, resources, and procedures to an agent	Is the task supported in a released feature, or planned for the future?
Qiskit Function	Takes structured input and performs a circuit or application task as a service	What are the access conditions, task scope, and result metrics?
Coding model or agent	Generates code and plans, selects tools, and calls them iteratively	What detects errors in the specification, implementation, and results?

These components play different roles. Qiskit Functions is currently in preview and is available to users on Premium, Flex, and On-Prem API plans. Access to a hosted service should not be equated with access to a public SDK. Qiskit's AI transpiler must also be distinguished from LLM coding assistance. The documented passes use reinforcement learning to improve resynthesis of supported circuit blocks or placement and routing. An agent can use that optimizer, but calling a tool does not replace its internal algorithm. [11,12]

The current status of each service matters as well. IBM's hosted Code Assistant preview ended on May 29, 2026, while guidance for using the public model locally remains available. In the pinned version of the official Skills repository examined here, Runtime code migration is a released task; broader problem mapping and optimization capabilities must be distinguished from the roadmap. Classiq, meanwhile, documents an agent interface and MCP and skills support for external coding tools. Its Quantum Engineering Interface specifies beta status and subscription conditions, while the Quantum Engineer Plugin specifies enterprise access. Comparing interface names alone mixes currently accessible features with future plans. [8,11,12]

The Qiskit QuantumKatas study compared models and prompt configurations across 350 tasks in 26 categories, using simulators to check the answers. In the category for solving SAT with Grover's algorithm, 55 of 160 attempts across 10 problems and 16 models succeeded, giving a success rate of **34.4%**. The comparison used the configuration with the best overall performance for each model. [13]

The practical differences between agents emerge in what happens next: whether an agent can use checking results to correct a wrong candidate, preserve both the high-level model and the generated circuit, and compare against small known-answer cases before moving to a larger calculation. An LLM's ability to call tools effectively must advance alongside the semantic, resource, and execution contracts those tools provide.

6. The two ways of partnering with hardware companies intersect

A company that develops both software and hardware can reflect its device's supported operations and execution constraints in its tools. A platform connecting multiple devices seeks to reduce the cost of trying different machines and the fragmentation of their interfaces. These approaches have different strengths, but dividing companies into two closed camps misses the connections that actually exist.

Qiskit is public software developed by IBM. Quantinuum owns hardware while also providing public TKET software and connections to other interfaces. Classiq's support for multiple backends and its public examples do not mean that its entire synthesis service and SDK are open source. NVIDIA provides CUDA-Q's public tools for connecting multiple QPUs alongside a GPU and execution infrastructure. **Hardware ownership, source openness, and portability across devices are three separate properties.** [5,10,14-16]

Connection	Integration identified in official sources	What the connection alone does not establish
Classiq → other circuit tools	Export to formats such as QASM; a function converting a Qmod program into a CUDA-Q kernel	Whether every program, feature, and device works without code changes
Qiskit → Quantinuum	Conversion to TKET circuits and submission to hardware through Nexus	Whether Qiskit exposes every device feature available through pytket
NVIDIA CUDA-Q	Hybrid Python and C++ programs, multiple QPUs, CPU/GPU simulation, and execution infrastructure	Whether QPU neutrality also removes dependencies on GPU environments and execution infrastructure
Amazon Braket	A cloud access point for multiple hardware systems and simulators	Whether device capabilities, prices, queues, and accuracy become identical
Azure Quantum and QIR	Provider-specific execution environments and a common intermediate representation between languages and targets	Whether all targets accepting the same representation offer the same features and performance

Classiq's CUDA-Q integration shows how the two approaches can be combined within one task. Classiq supplies the higher-level model and synthesis, while CUDA-Q supplies execution, simulation, and hybrid tools. Qiskit also connects to other hardware. Quantinuum's official guidance provides an example of converting and submitting a Qiskit circuit through TKET, while noting that this route does not guarantee access to every device feature. Supporting a connection is different from claiming that every feature transfers without loss. [5,15]

Efforts to establish a common intermediate representation such as QIR reduce the duplication involved in building a separate translator for every new language and new device. Supported operations, control flow, placement and routing, error correction, and execution systems still sit underneath. Porting while preserving high-level meaning, and porting while preserving resource use and physical performance, each require testing. The number of partner logos may indicate the breadth of connections, but it does not measure performance or sustained usage. [16-18]

7. Does an easier route eliminate a technology—or change its position?

When a new tool is costly to learn, developers may choose to combine familiar Python code and existing libraries. If an agent performs that composition, the cost of building an alternative route also falls. This creates competitive pressure that specialized synthesis platforms need to address. If another tool delivers the same result with less effort, users have a reason to choose it regardless of the model it uses.

At the same time, an agent can also reduce the effort of learning Qmod. In one possible arrangement, a user makes a natural-language request and an agent constructs the model and calls a specialized synthesis engine. The agent provides the user-facing interface, while the specialized engine selects the circuit implementation behind it. The quality and computation cost of the circuits it produces become concrete criteria for choosing that engine.

Three areas may retain meaningful differences. The first is how well a system handles difficult implementation choices and resource tradeoffs. The second is how effectively experience with failures, verification, and execution becomes reusable tests and improvements to tools. The third is how smoothly it preserves cost, version, and result records within customers' actual workflows. Public tools can accumulate these capabilities, and commercial services must provide them to demonstrate a meaningful distinction.

The code and verification records accumulated in an existing ecosystem are less conspicuous than a new technology, but difficult to migrate. Conversely, if knowing the syntax of a particular language is the only barrier, an agent can lower it quickly. The durability of an advantage should therefore be assessed through the reproducible outcomes delivered behind the interface, rather than through the imposition of an unfamiliar interface on users.

8. Observations that distinguish three possible futures

We give greater weight to a hybrid structure for the future of quantum software, with agents at the entry point and specialized synthesis engines and execution platforms working together behind them. The three paths below identify the conditions that would strengthen each role and the results by which to assess its competitiveness.

Conditional outlook	Conditions that would strengthen it	Observations that could change the assessment
Public tools combined with agents become the default working environment	Frequently used functions, checks, and target transformations are sufficient, with low integration and maintenance costs	Expert intervention time, total cost of successful tasks, and modifications required to move to a new target
Specialized synthesis engines remain essential tools behind agents	They produce competitive circuits for large arithmetic operations, oracles, and resource reuse, even after accounting for substantial classical search costs	Comparisons using current versions, equal precision, and equal compilation time; reproducible width-operation tradeoffs
Integration with hardware and execution infrastructure becomes more valuable	Applications couple closely to actual device control, real-time feedback, and error correction	Execution latency and data movement, error-correction costs, and the accuracy, retries, and operating conditions of actual results

Classiq-CUDA-Q and Qiskit-TKET are existing examples of connections between different roles. Both approaches are also expanding capabilities that agents can invoke. These connections fit an arrangement in which a general-purpose agent serves as the entry point, specialized synthesis handles circuit implementation, and an execution platform manages resource access and operations. In this arrangement, the brand visible to the user may differ from the supplier of the technology that reduces computation costs. If combinations of open tools repeatedly match specialized synthesis under the same budget, the differentiation of that specialized layer would diminish.

In the nearer term, automation of documentation search and coding may spread rapidly. Differences confined to natural-language interfaces would then narrow, while supported capabilities and procedures for correcting failures become more visible. In larger quantum algorithms, the cost of arithmetic, state preparation, and oracles may increase the need for specialized synthesis. Yet high-level functions and synthesis plugins in existing SDKs will also develop, so none of these areas can be assigned in advance as an exclusive domain of one company.

Fault-tolerant computation requires more than logical qubit and CX counts. In representative surface-code-based implementations, application cost also depends on the supply of additional resources such as the magic states consumed for T operations, error-correcting codes and device layout, and the time required for classical decoders to infer errors from measurements and for control operations. At this stage, the more useful architecture connects verified tools and execution layers accurately, rather than imagining an LLM directly

replacing a fast physical control loop. CUDA-Q's hybrid tools and Classiq's fault-tolerant resource-estimation features each address parts of this direction, but supported features, previews, and model-based estimates must be distinguished from results achieved on actual devices. [5,14,18]

9. The next comparison must count the full cost of one successful calculation

A more persuasive comparison starts from the same problem definition and oracle, numerical precision, and permitted error. It records current versions of each tool, the available functions and plugins, and the target's operation and connectivity constraints, then matches the time budgets for design and compilation. A circuit obtained through a longer synthesis search and a circuit produced in one agent attempt should not be counted as having the same cost.

Next, small known-answer cases and equivalence or state checks establish whether the program implements the intended condition. Logical-circuit metrics are distinguished from metrics after device-specific transformation. On physical devices, the record includes shots, error mitigation, retries, queues, and data movement. It must retain failed tasks and the time spent on human corrections, as well as average performance. This does not imply that every check is easy to perform on a large problem. The report must disclose which parts were checked exactly and which were estimated through samples or models.

The resulting basis for comparison is closer to “What was the total cost of obtaining one result that passed the required conditions?” than to “How many lines of code were saved?” Whether a system successfully completes more tasks within the same resource budget, and maintains those results across other targets and later versions, tests the technical capabilities it accumulates.

Returning to the opening `101` makes the competition in quantum software more concrete. Defining the right answer, deciding how long to preserve that meaning, trading space against operations, and checking the results obtained from a real device are connected responsibilities. High-level synthesis and agent-driven tool composition reduce the burden at different points and can also work together. Shifts in platform power should be judged through the differences in quality, cost, and user experience that remain after following those connections all the way through.

References

[1] M. Vax, P. Emanuel, E. Cornfeld, I. Reichental, O. Opher, O. Roth, T. Michaeli, L. Preminger, L. Gazit, A. Naveh, and Y. Naveh, Qmod: Expressive high-level quantum programming, in *2025 IEEE International Conference on Quantum Software (QSW)* (IEEE, 2025), pp. 141-150.

[2] Classiq, Qmod - the quantum modeling language (accessed 26 October 2026); Classiq, Quantum numbers and arithmetics (accessed 26 October 2026); Classiq, Uncomputation (accessed 26 October 2026); Classiq, Quantum variables (accessed 26 October 2026); Classiq, Quantum program synthesis (accessed 26 October 2026); Classiq, Quantum program transpilation (accessed 26 October 2026).

[3] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, Quantum computing with Qiskit, arXiv:2405.08810v3; Qiskit, version 2.6.1, GitHub (2026), commit 2c4e509; Qiskit, hls_plugins.py (2026); Qiskit, high_level_synthesis.py (2026); IBM Quantum, Circuit synthesis (accessed 26 October 2026); IBM Quantum, TGate (accessed 26 October 2026).

[4] IBM Quantum, Target (accessed 26 October 2026); IBM Quantum, Transpiler (accessed 26 October 2026).

[5] Classiq, CUDA-Q integration (accessed 26 October 2026); Classiq, Synthesis (accessed 26 October 2026); Classiq, Supported cloud providers (accessed 26 October 2026).

[6] L. K. Grover, A fast quantum mechanical algorithm for database search, in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing* (ACM, 1996), pp. 212–219.

[7] AI Tech Review, AI_Tech_Review, GitHub (2026), commit d208e87.

[8] Classiq, Quantum Engineering Interface (accessed 26 October 2026); Classiq, Quantum Engineer Plugin (accessed 26 October 2026); Classiq, AI Agent in Studio (accessed 26 October 2026).

[9] T. Goldfriend *et al.*, Design and synthesis of scalable quantum programs, arXiv:2412.07372v2.

[10] Classiq Technologies, classiq, version 1.33.1 (2026); Classiq, Classiq: End to end platform for developing quantum software, GitHub (2026), commit 2bfd835; Classiq, LICENSE.txt (2026); Classiq, Changelog (accessed 26 October 2026); Classiq, Quantum program constraints (accessed 26 October 2026).

[11] Qiskit, Qiskit MCP servers, GitHub (2026), commit b763581; Qiskit, Qiskit AI skills, GitHub (2026), commit a5499c7; IBM Quantum, Introduction to Qiskit Functions (accessed 26 October 2026); IBM Quantum, Qiskit Code Assistant (accessed 26 October 2026); IBM Quantum, IBM Quantum Compute Service changelog (accessed 26 October 2026).

[12] D. Kremer, V. Villar, H. Paik, I. Duran, I. Faro, and J. Cruz-Benito, Practical and efficient quantum circuit synthesis and transpiling with reinforcement learning, arXiv:2405.13196v2; IBM Quantum, AI-powered transpiler passes (accessed 26 October 2026); Qiskit, qiskit_ibm_transpiler, GitHub (2026), commit 3aa0a92.

[13] J. Cruz-Benito and I. Faro, Qiskit QuantumKatas: Adapting Microsoft's quantum computing exercises for LLM evaluation, arXiv:2605.27210v1; Quantum Katas Benchmark, GitHub (2026), commit f930496.

[14] The CUDA-Q development team, CUDA-Q, GitHub (2026), commit de8b705; NVIDIA, CUDA-Q (accessed 26 October 2026).

[15] Quantinuum, Using Qiskit with Quantinuum devices (accessed 26 October 2026).

[16] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan, t|ket⟩: A retargetable compiler for NISQ devices, Quantum Sci. Technol. **6**, 014003 (2021); Quantinuum, tket, GitHub (2026), commit 4a554cb; Quantinuum, What is tket? (accessed 26 October 2026).

[17] Amazon Web Services, What is Amazon Braket? (accessed 26 October 2026).

[18] Microsoft, Quantum computing providers on Azure Quantum (accessed 26 October 2026); Microsoft, Quantum intermediate representation (accessed 26 October 2026); Classiq, Fault-tolerance engine (accessed 26 October 2026).

How this review was made

Responsible editor	Hyun-Jung Kim
AI assistance	OpenAI Codex; exact authoring-model identifier not retained.
Agents and roles	root: Korean writing, original conceptual cover, NumPy teaching calculation, ecosystem research and publication integration; academic_sources: current Qiskit HLS, Target, agent tools and scientific audit; academic_sources/theory_crosscheck: LLM benchmark denominators and claim-scope review; applications_sources: Classiq papers, SDK, guarantees, reported-data crosscheck and full English translation; applications_sources/translation_audit: independent bilingual audit of sources, notation, equations and conditions; review_format: original bilingual figures, equation/font typesetting, HTML/PDF production and output checks; publish_audit: independent explanation and conceptual-cover review, and publication audit.
Inputs and outputs	Shared material, public papers, official SDK documentation/code and a NumPy teaching calculation → bilingual review, conceptual cover, explanatory figures/equations and PDFs.
Production request	Review Classiq and Qiskit approaches to quantum synthesis and circuit transformation, agent tool use and verification, and choices in the quantum-software ecosystem.
Harness	AI Tech Review Editorial Harness 2026.08; Scientific Explanation and Visual Communication Principles 1.0 (2026-10-22)
Verification scope	Primary sources, versions, SDK documentation and paper-reported values; a NumPy calculation of three-qubit phase oracles and one Grover iteration; bilingual equations, figures and HTML/PDF output. The Classiq synthesis results were not independently reproduced. No account, paid API or QPU execution.
Human review	Topic, writing and public posting requested; no line-by-line human approval.
Evidence cutoff	2026-10-25
Prose revision	Prose revision, 2026-10-26: five Korean paragraphs and their English counterparts were edited using the October 26 prose-audit harness supplement. root handled editing, output and publication integration; english_update reviewed the English correspondence; publication_audit reviewed publication consistency. Numerical values, equations, figures, references and the evidence cutoff were retained.
Reference revision	APS reference revision, 2026-10-26: root handled citation mapping, typesetting and publication; aps_standard_audit checked the official style; software_credit_metadata checked software attribution; bibliography_source_index verified official documentation titles.

The teaching state-vector calculation, author-reported synthesis values and conceptual diagrams are distinguished. The AI-generated cover is conceptual, not an actual device, circuit or benchmark dataset.